Normalization Kata

Case 1 – Movies table with the id column removed

The problem and its possible impact:
Without an ID column, the table lacks a Primary Key. This makes it impossible to uniquely identify a specific row. If two movies have the exact same title and release year, the database cannot tell them apart. Note that the removal creates duplication though no new records are added. Furthermore, reliable Foreign Key relationships cannot be established.

How to solve the problem?
The best solution is to avoid the change and keep the existing Primary Key. If the ID column was removed, a composite key based on attributes such as movie name, year, and director can be used to uniquely identify movies.

Is it a normalization rule violation?
Yes. This violates First Normal Form (1NF), because after removing the primary key, rows that were previously different only because of their ID may become identical. Therefore, duplicate records can exist and rows are no longer uniquely identifiable.

Case 2 – Movies table with the id column duplicated

The problem and its possible impact:

Having two columns that store the same ID value provides no additional information and creates unnecessary redundancy. It wastes storage space and may lead to inconsistencies if one column is updated while the other is not. In addition, a key that includes duplicated information is not minimal, since one of the columns can be removed without affecting the ability to uniquely identify a record.

How to solve the problem?
Avoid duplicating the ID column. The same information should be stored only once in the table. If a duplicate ID column already exists, it should be removed so that only one ID column remains.

Is it a normalization rule violation?

This is a semantics question. Normalization conditions assume the use of a **single** key. When the ID is duplicated, both columns can serve as keys, and if they are considered together they form a non-minimal key. Therefore, whether this should be considered a normalization violation is largely a matter of interpretation, and both claims can be justified.

Case 3 – Table columns in Hebrew (e.g., "shem", "shana")

The problem and its possible impact:
Using Hebrew or transliterated Hebrew names for table columns reduces readability and consistency in the database design. It can make the schema harder to understand, maintain, and use, especially when working with people who are not familiar with Hebrew.

How to solve the problem?
Use standard English naming conventions for all database objects. Hebrew should only appear in the user interface layer if needed.

Is it a normalization rule violation?
No. This is a naming convention and system design issue rather than a normalization issue.

Case 4 – Directors column in movies

The problem and its possible impact:
Many movies may have multiple directors. Storing all directors inside a single column forces multiple values into one cell, usually as a comma-separated list. This makes querying and maintaining the data inefficient and difficult.In addition, directors represent a separate entity that can be related to many movies. Storing directors directly in the Movies table creates redundancy and does not properly represent the relationship between movies and directors.

How to solve the problem?
Create a separate Directors table and a Movies_Directors junction table, as is done in the IMDB schema, to represent the many-to-many relationship between movies and directors.

Is it a normalization rule violation?
Yes. This violates First Normal Form (1NF), which requires each column to contain atomic values only.

Case 5 – The minimal id of a playing actor column in movies table

The problem and its possible impact:

The minimal_actor_id value is derived from the relationship between movies and actors rather than being direct information about the movie itself. While storing calculated values can sometimes improve performance, the minimal actor ID is an internal system value that has little business meaning and is unlikely to be useful to users. As a result, storing it creates unnecessary redundancy and may lead to inconsistencies if actor assignments change.

How to solve the problem?

Remove the minimal_actor_id column. Since this value has little business meaning and can be calculated when needed, there is no benefit in storing it in the Movies table.

Is it a normalization rule violation?

No. The minimal actor ID is derived from data related to the movie, so it is still functionally dependent on the movie. The issue is not a normalization violation but rather that the value has little practical meaning, can be calculated when needed, and may become inconsistent if the actor list changes.

Case 6 – A constant field (e.g., Pi) in movies table

The problem and its possible impact:

A constant value such as Pi is unrelated to the Movies entity and does not represent any property of a movie. Including unrelated attributes makes the schema less meaningful and wastes storage. In addition, the same value is unnecessarily repeated in every row.

How to solve the problem?

Remove the column from the table. If the value is needed globally, it should be stored separately or calculated when necessary.

Is it a normalization rule violation?

No. Although Pi is unrelated to the Movie entity, it does not create a functional dependency problem. Given a movie key, the value of Pi is always uniquely determined (the same constant value for every record). Therefore, the normalization requirement that attributes be functionally dependent on the key is still satisfied.

Note that Pi being a constant is not the key aspect. As long as we can deduce a property from the key, we have functional dependency.

Case 7 – Having id number and employee number in employees table

The problem and its possible impact:
Having both an ID Number and an Employee Number is often useful and desirable. The Employee Number can be used internally within the organization (for example, to identify employees and their departments), while the ID Number can be used for external or legal purposes such as reporting to government authorities. However, if both identifiers are used inconsistently across the database, confusion and maintenance difficulties may occur.

How to solve the problem?
Keep both identifiers, since they serve different purposes. Define clear guidelines for their usage. For example, Employee Number can be used for internal database operations and relationships, while ID Number can be used for external and legal purposes. Consistent use of each identifier helps prevent confusion and maintenance issues.

Is it a normalization rule violation?

This is a semantics question.

From a normalization perspective, this case is similar to duplicating the ID column. The table contains two identifiers, each of which can uniquely identify an employee. However, unlike a duplicated ID column, the two identifiers serve different business purposes. The Employee Number is used internally within the organization, while the ID Number is used for external and legal purposes. Therefore, even if we consider the existence of two keys to be normalization violation, there is a practical justification for keeping both attributes.